

DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT 25CS1302E

WEEK 8 – PRODUCT CRUD USING SPRING BOOT + MYSQL + FRONTEND

1. AIM

To develop a Product CRUD REST API using Spring Boot, Spring Data JPA and MySQL, and connect it with a frontend using HTML, CSS and JavaScript.

****CRUD operations:****

1. Create a product
2. Read product details
3. Update a product
4. Delete a product

2. REQUIREMENTS

****Software:****

- * Spring Tools Suite (STS)
- * Java
- * Spring Boot
- * MySQL
- * MySQL Workbench
- * Postman
- * VS Code
- * Web Browser

****Technologies:****

- * Spring Web
- * Spring Data JPA
- * MySQL Driver
- * HTML
- * CSS
- * JavaScript

3. PROJECT STRUCTURE

Backend

```
``text
week8_co3
|
```

```

|----- src/main/java
|
|   |----- com.klu.week8_co3
|   |
|   |   |----- entity
|   |   |   |----- Product.java
|   |   |----- repository
|   |   |   |----- ProductRepository.java
|   |   |----- service
|   |   |   |----- ProductService.java
|   |   |   |----- ProductServiceImpl.java
|   |   |----- controller
|   |   |   |----- ProductController.java
|
|----- src/main/resources
|   |----- application.properties
...

```

Frontend

```

```text
product-frontend
|
|----- index.html
|----- style.css
|----- script.js
...

```

---

## # 4. DATABASE SETUP

**\*\*Database Name:\*\*** `kluh`

**\*\*Table Name:\*\*** `products`

### ### SQL

```

```sql
CREATE DATABASE kluh;

USE kluh;

CREATE TABLE products (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(255) NOT NULL,
  description TEXT,
  price DECIMAL(10,2) NOT NULL,
  quantity INT NOT NULL
);
...

```

Sample Data

```

```sql
INSERT INTO products (name, description, price, quantity) VALUES
('Laptop', 'Dell XPS 13', 1200.00, 10),
('Smartphone', 'Samsung Galaxy S21', 999.00, 20),
('Tablet', 'Apple iPad Pro', 799.00, 15);
```

```

To check the data

```

```sql
SELECT * FROM products;
```

```

5. APPLICATION.PROPERTIES

```

```properties
spring.application.name=week8_co3

server.port=8089

spring.datasource.url=jdbc:mysql://localhost:3306/kluh
spring.datasource.username=root
spring.datasource.password=YOUR_MYSQL_PASSWORD

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

```

****Note:**** Replace `YOUR\_MYSQL\_PASSWORD` with your actual MySQL password.

6. PRODUCT ENTITY – Product.java

```

```java
package com.klu.week8_co3.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "products")
public class Product {

 @Id
 @GeneratedValue(strategy = GenerationType.IDENTITY)
 private Long id;

 private String name;
 private String description;
 private double price;
 private int quantity;

 public Product() {
 }

 public Product(String name, String description,
 double price, int quantity) {

```

```

 this.name = name;
 this.description = description;
 this.price = price;
 this.quantity = quantity;
 }

 public Long getId() {
 return id;
 }

 public void setId(Long id) {
 this.id = id;
 }

 public String getName() {
 return name;
 }

 public void setName(String name) {
 this.name = name;
 }

 public String getDescription() {
 return description;
 }

 public void setDescription(String description) {
 this.description = description;
 }

 public double getPrice() {
 return price;
 }

 public void setPrice(double price) {
 this.price = price;
 }

 public int getQuantity() {
 return quantity;
 }

 public void setQuantity(int quantity) {
 this.quantity = quantity;
 }
}

```

---

## # 7. PRODUCT REPOSITORY – ProductRepository.java

```

```java
package com.klu.week8_co3.repository;

import com.klu.week8_co3.entity.Product;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository

```

```
public interface ProductRepository
    extends JpaRepository<Product, Long> {
}
---
```

8. PRODUCT SERVICE – ProductService.java

```
```java
package com.klu.week8_co3.service;

import com.klu.week8_co3.entity.Product;
import java.util.List;

public interface ProductService {

 List<Product> getAllProducts();

 Product getProductById(Long id);

 Product saveProduct(Product product);

 Product updateProduct(Long id, Product product);

 void deleteProduct(Long id);
}

```

#### # 9. PRODUCT SERVICE IMPLEMENTATION – ProductServiceImpl.java

```
```java
package com.klu.week8_co3.service;

import com.klu.week8_co3.entity.Product;
import com.klu.week8_co3.repository.ProductRepository;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
public class ProductServiceImpl implements ProductService {

    private final ProductRepository productRepository;

    public ProductServiceImpl(ProductRepository productRepository) {
        this.productRepository = productRepository;
    }

    @Override
    public List<Product> getAllProducts() {
        return productRepository.findAll();
    }

    @Override
    public Product getProductById(Long id) {
        return productRepository.findById(id)
            .orElseThrow(() ->

```

```

        new RuntimeException("Product not found"));
    }

    @Override
    public Product saveProduct(Product product) {
        return productRepository.save(product);
    }

    @Override
    public Product updateProduct(Long id,
                                Product productDetails) {

        Product existingProduct = getProductById(id);

        existingProduct.setName(productDetails.getName());

        existingProduct.setDescription(
            productDetails.getDescription());

        existingProduct.setPrice(productDetails.getPrice());

        existingProduct.setQuantity(
            productDetails.getQuantity());

        return productRepository.save(existingProduct);
    }

    @Override
    public void deleteProduct(Long id) {
        productRepository.deleteById(id);
    }
}

```

10. PRODUCT CONTROLLER – ProductController.java

```

```java
package com.klu.week8_co3.controller;

import com.klu.week8_co3.entity.Product;
import com.klu.week8_co3.service.ProductService;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@CrossOrigin(origins = "**")
@RequestMapping("/api/products")
public class ProductController {

 private final ProductService productService;

 public ProductController(ProductService productService) {

 this.productService = productService;
 }
}

```

```

 }

 @GetMapping
 public List<Product> getAllProducts() {

 return productService.getAllProducts();

 }

 @GetMapping("/{id}")
 public ResponseEntity<Product> getProductById(
 @PathVariable Long id) {

 return ResponseEntity.ok(
 productService.getProductById(id));

 }

 @PostMapping
 public ResponseEntity<Product> createProduct(
 @RequestBody Product product) {

 return ResponseEntity.ok(
 productService.saveProduct(product));

 }

 @PutMapping("/{id}")
 public ResponseEntity<Product> updateProduct(
 @PathVariable Long id,
 @RequestBody Product productDetails) {

 return ResponseEntity.ok(
 productService.updateProduct(id, productDetails)
);

 }

 @DeleteMapping("/{id}")
 public ResponseEntity<Void> deleteProduct(
 @PathVariable Long id) {

 productService.deleteProduct(id);

 return ResponseEntity.noContent().build();

 }
}

```

---

## # 11. WHY @CrossOrigin IS USED

```

```java
@CrossOrigin(origins = "**")
```

```

It allows the frontend to communicate with the Spring Boot REST API when the frontend and backend run on different origins or ports.

**\*\*Backend API:\*\***

```
```text
http://localhost:8089/api/products
```
```

---

## # 12. HOW TO START BACKEND

1. Open `week8\_co3` in STS.
2. Open `Week8Co3Application.java`.
3. Right-click the file.
4. Select **\*\*Run As → Spring Boot App\*\***.
5. Wait for:

```
```text
Tomcat started on port 8089
```
```

6. Keep Spring Boot running while using the frontend.

### To stop Spring Boot

\* Click the red **\*\*Stop/Terminate\*\*** button in the Console.

OR

\* Press:

```
```text
Command + F2
```
```

---

## # 13. API TESTING USING POSTMAN

### BASE URL

```
```text
http://localhost:8089/api/products
```
```

### A) GET ALL PRODUCTS

**\*\*Method:\*\*** GET

```
```text
http://localhost:8089/api/products
```
```

**\*\*Purpose:\*\*** Displays all products.

**\*\*Result:\*\*** 200 OK

---

### B) GET PRODUCT BY ID

**\*\*Method:\*\* GET**

```
```text
http://localhost:8089/api/products/2
```
```

**\*\*Purpose:\*\* Displays the product with ID 2.**

**\*\*Result:\*\* 200 OK**

---

### ### C) CREATE PRODUCT

**\*\*Method:\*\* POST**

```
```text
http://localhost:8089/api/products
```
```

**\*\*Body → raw → JSON\*\***

```
```json
{
  "name": "Keyboard",
  "description": "Mechanical Keyboard",
  "price": 2500,
  "quantity": 5
}
```
```

**\*\*Result:\*\* Product created successfully.**

---

### ### D) UPDATE PRODUCT

**\*\*Method:\*\* PUT**

```
```text
http://localhost:8089/api/products/1
```
```

**\*\*Body → raw → JSON\*\***

```
```json
{
  "name": "Gaming Laptop",
  "description": "Dell Gaming Laptop",
  "price": 1500,
  "quantity": 8
}
```
```

**\*\*Result:\*\* Product updated successfully.**

---

### ### E) DELETE PRODUCT

**\*\*Method:\*\* DELETE**

```
```text
http://localhost:8089/api/products/1
```
```

**\*\*Result:\*\*** Product deleted successfully.

---

## # 14. FRONTEND SETUP

Create a folder:

```
```text
product-frontend
```
```

Inside the folder create:

```
```text
index.html
style.css
script.js
```
```

---

## # 15. index.html

```
```html
<!DOCTYPE html>
<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport"
    content="width=device-width, initial-scale=1.0">

  <title>Product Management</title>

  <link rel="stylesheet" href="style.css">

</head>

<body>

  <div class="container">

    <h1>Product Management System</h1>

    <div class="form-box">

      <h2>Add Product</h2>

      <input type="text"
        id="name"
```
```

```

 placeholder="Product Name">

<input type="text"
 id="description"
 placeholder="Description">

<input type="number"
 id="price"
 placeholder="Price">

<input type="number"
 id="quantity"
 placeholder="Quantity">

<button onclick="addProduct()">
 Add Product
</button>

</div>

<h2>Products</h2>

<table>

 <thead>

 <tr>

 <th>ID</th>
 <th>Name</th>
 <th>Description</th>
 <th>Price</th>
 <th>Quantity</th>
 <th>Action</th>

 </tr>

 </thead>

 <tbody id="productTable">

 </tbody>

</table>

</div>

<script src="script.js"></script>

</body>

</html>

```

---

# 16. style.css

```

css
body {

```

```
 font-family: Arial, sans-serif;
 background: #f2f2f2;
 margin: 0;
 padding: 20px;
}
```

```
.container {
 max-width: 1000px;
 margin: auto;
 background: white;
 padding: 30px;
 border-radius: 10px;
}
```

```
h1 {
 text-align: center;
}
```

```
.form-box {
 margin-bottom: 30px;
}
```

```
input {
 padding: 10px;
 margin: 5px;
 width: 200px;
}
```

```
button {
 padding: 10px 15px;
 margin: 5px;
 cursor: pointer;
}
```

```
table {
 width: 100%;
 border-collapse: collapse;
}
```

```
th, td {
 border: 1px solid #ccc;
 padding: 10px;
 text-align: center;
}
```

```
th {
 background: #eee;
}
...
```

---

# 17. script.js

```
````javascript
const API_URL = "http://localhost:8089/api/products";
```

```
// GET PRODUCTS
```

```

function loadProducts() {
  fetch(API_URL)
    .then(response => response.json())
    .then(products => {
      const table =
        document.getElementById("productTable");
      table.innerHTML = "";
      products.forEach(product => {
        table.innerHTML += `
          <tr>
            <td>${product.id}</td>
            <td>${product.name}</td>
            <td>${product.description}</td>
            <td>${product.price}</td>
            <td>${product.quantity}</td>
            <td>
              <button
                onclick="deleteProduct(${product.id})">
                Delete
              </button>
            </td>
          </tr>
        `;
      });
    })
    .catch(error => {
      console.error("Error:", error);
      alert("Could not connect to Spring Boot");
    });
}

```

// ADD PRODUCT

```

function addProduct() {
  const product = {
    name:
      document.getElementById("name").value,
    description:
      document.getElementById("description").value,
    price:
      document.getElementById("price").value,
    quantity:
      document.getElementById("quantity").value
  };
  fetch(API_URL, {
    method: "POST",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify(product)
  })
  .then(response => response.json())
  .then(() => {
    alert("Product Added Successfully!");
    loadProducts();
  });
}

```

// DELETE PRODUCT

```

function deleteProduct(id) {
  fetch(`${API_URL}/${id}`, {
    method: "DELETE"
  })
  .then(() => {
    alert("Product Deleted!");
    loadProducts();
  });
}

```

```
});  
}
```

```
// LOAD PRODUCTS WHEN PAGE OPENS
```

```
loadProducts();  
---
```

18. HOW FRONTEND CONNECTS TO BACKEND

```
```text  
FRONTEND
HTML + CSS + JavaScript
 |
 | JavaScript fetch()
 ↓
SPRING BOOT REST CONTROLLER
 |
 ↓
SERVICE LAYER
 |
 ↓
JPA REPOSITORY
 |
 ↓
MYSQL DATABASE
```
```

****API URL.****

```
```text  
http://localhost:8089/api/products
```
```

19. HOW TO RUN FRONTEND

1. Open `product-frontend` in VS Code.
2. Make sure these three files are in the same folder:

```
* `index.html`  
* `style.css`  
* `script.js`
```

3. Open `index.html` using ****Live Server****.
4. Keep Spring Boot running in STS.
5. The browser should display the products fetched from the backend.

20. FRONTEND OPERATIONS

VIEW PRODUCTS

When the page opens, JavaScript calls the GET API and displays the products in the table.

ADD PRODUCT

Enter:

- * Product Name
- * Description
- * Price
- * Quantity

Click ****Add Product****.

The frontend sends a POST request to Spring Boot.

DELETE PRODUCT

Click the ****Delete**** button.

The frontend sends a DELETE request to Spring Boot.

The database is updated automatically.

21. OBSERVATION

The Product Management System was successfully connected with MySQL.

CRUD operations were tested using Postman and the frontend.

| Operation | Method | Status |
|-----------|--------|---------|
| ----- | ----- | ----- |
| Create | POST | Success |
| Read | GET | Success |
| Update | PUT | Success |
| Delete | DELETE | Success |

22. RESULT

Thus, the Product CRUD REST API was successfully developed using Spring Boot, Spring Data JPA and MySQL.

The frontend was successfully created using HTML, CSS and JavaScript and connected to the Spring Boot REST API.

All CRUD operations were successfully tested.

23. VIVA QUESTIONS

****1. What is CRUD?****

CRUD stands for Create, Read, Update and Delete.

****2. What is Spring Boot?****

Spring Boot is a framework used to develop Java-based applications easily.

****3. What is REST API?****

REST API allows applications to communicate using HTTP methods.

****4. What is JPA?****

JPA is used to map Java objects to database tables.

****5. What is MySQL?****

MySQL is a relational database management system.

****6. Why is @RestController used?****

It is used to create REST API controllers.

****7. Why is @CrossOrigin used?****

It allows the frontend to communicate with the backend from a different origin.

****8. What is @GetMapping?****

It handles HTTP GET requests.

****9. What is @PostMapping?****

It handles HTTP POST requests.

****10. What is @PutMapping?****

It handles HTTP PUT requests.

****11. What is @DeleteMapping?****

It handles HTTP DELETE requests.

****12. What is JpaRepository?****

It provides ready-made database operations such as save, find and delete.

****13. What is the backend port?****

The backend is running on port ****8089****.

****14. What is the API endpoint?****

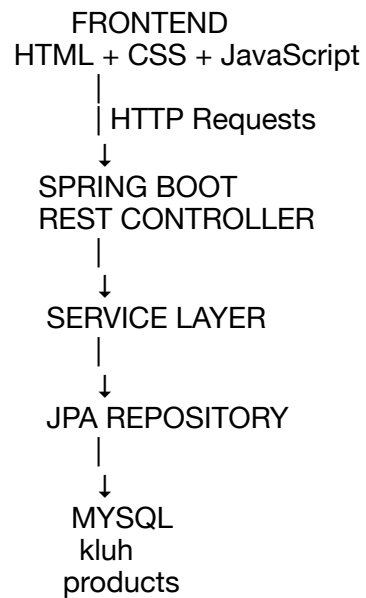
```
```text
http://localhost:8089/api/products
```
```

****15. Which technologies are used in the frontend?****

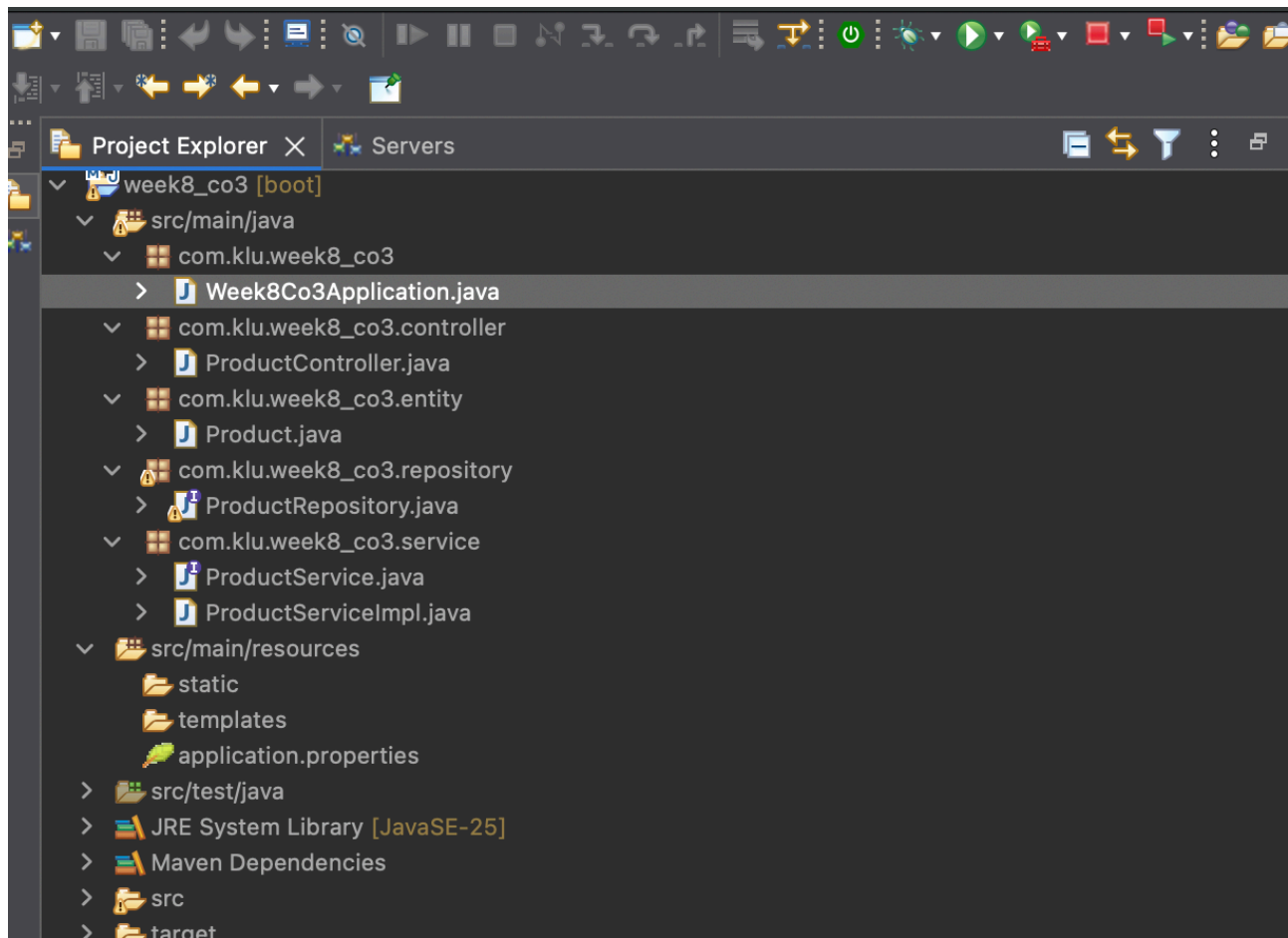
HTML, CSS and JavaScript.

24. FINAL ARCHITECTURE

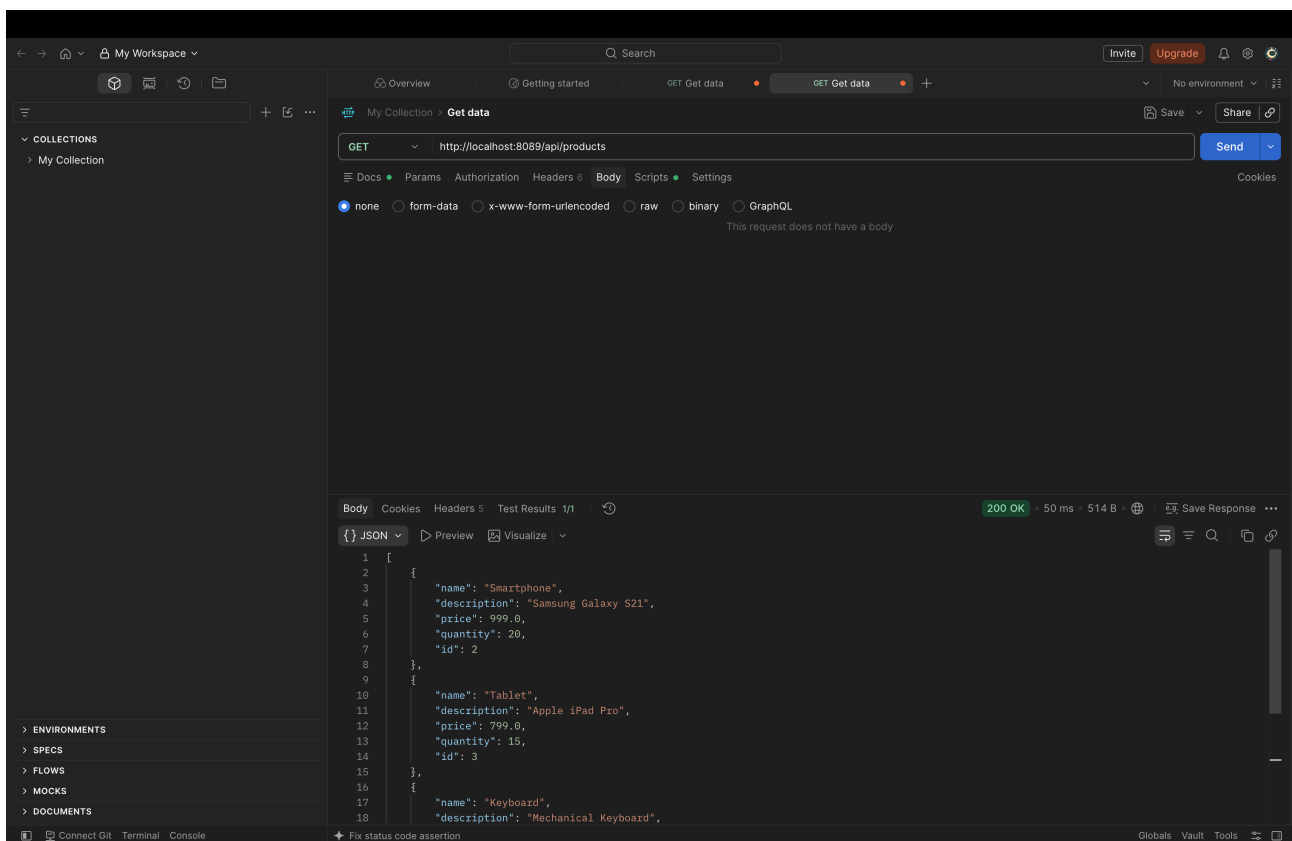
```
```text
```

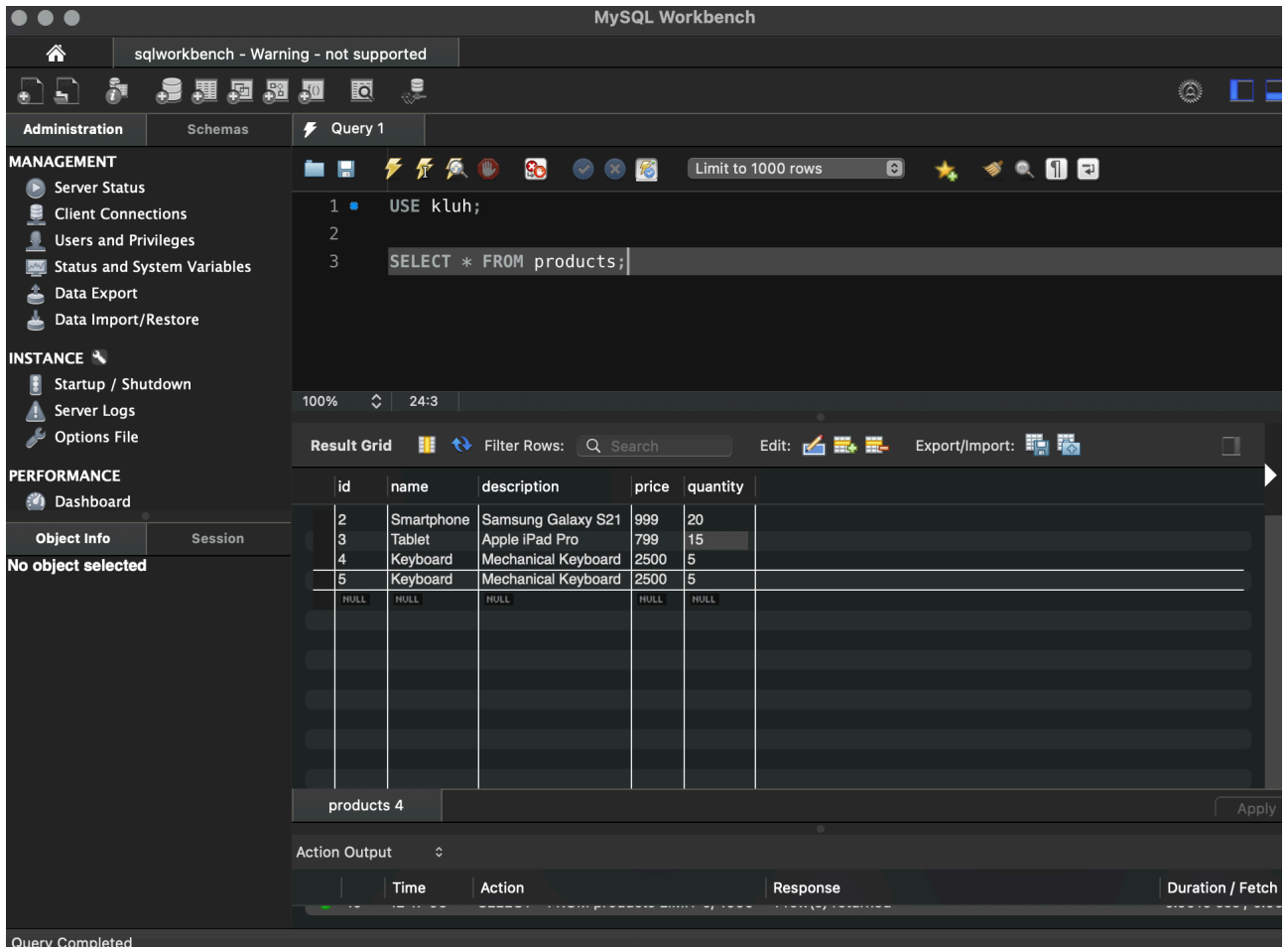


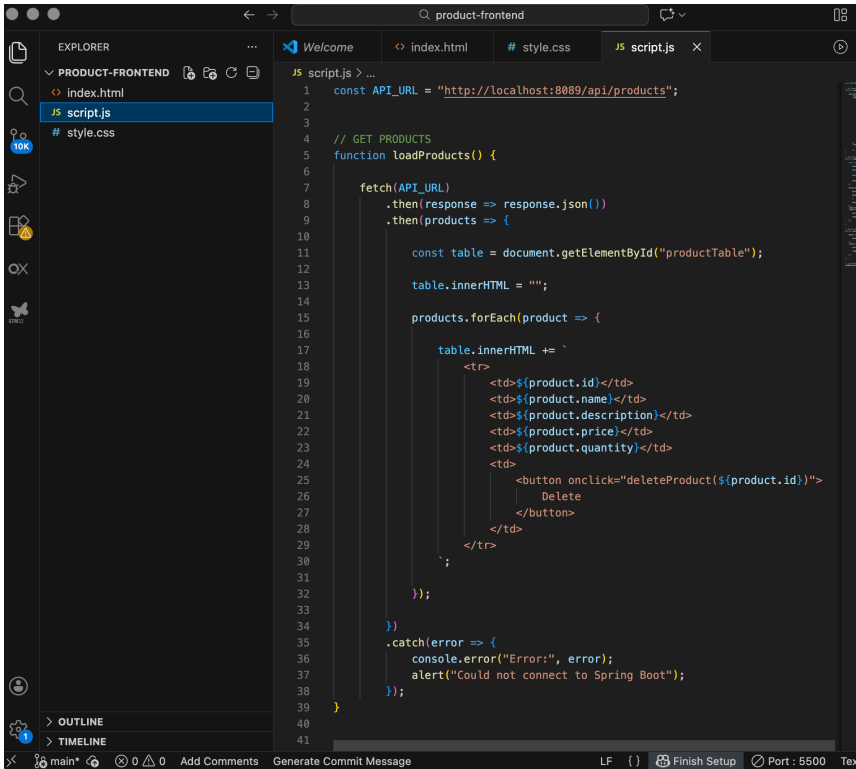
# END OF PRACTICAL



```
1 package com.klu.week8_co3;
2
3 import org.springframework.boot.SpringApplication;
4
5
6 @SpringBootApplication
7 public class Week8Co3Application {
8
9 public static void main(String[] args) {
10 SpringApplication.run(Week8Co3Application.class, args);
11 }
12 }
13
14
```







The image shows a VS Code editor window with the file explorer on the left displaying a project named 'PRODUCT-FRONTEND' containing 'index.html', 'style.css', and 'script.js'. The 'script.js' file is open in the editor, showing the following code:

```
1 const API_URL = "http://localhost:8089/api/products";
2
3
4 // GET PRODUCTS
5 function loadProducts() {
6
7 fetch(API_URL)
8 .then(response => response.json())
9 .then(products => {
10
11 const table = document.getElementById("productTable");
12
13 table.innerHTML = "";
14
15 products.forEach(product => {
16
17 table.innerHTML += `
18 <tr>
19 <td>${product.id}</td>
20 <td>${product.name}</td>
21 <td>${product.description}</td>
22 <td>${product.price}</td>
23 <td>${product.quantity}</td>
24 <td>
25 <button onclick="deleteProduct(${product.id})">
26 Delete
27 </button>
28 </td>
29 </tr>
30 `;
31 });
32 });
33 }
34
35 .catch(error => {
36 console.error("Error:", error);
37 alert("Could not connect to Spring Boot");
38 });
39
40
41
```

